

IC526 Advanced Compilers

Lec 12: IR Generation

Yoonseung Kim

DGIST EECS

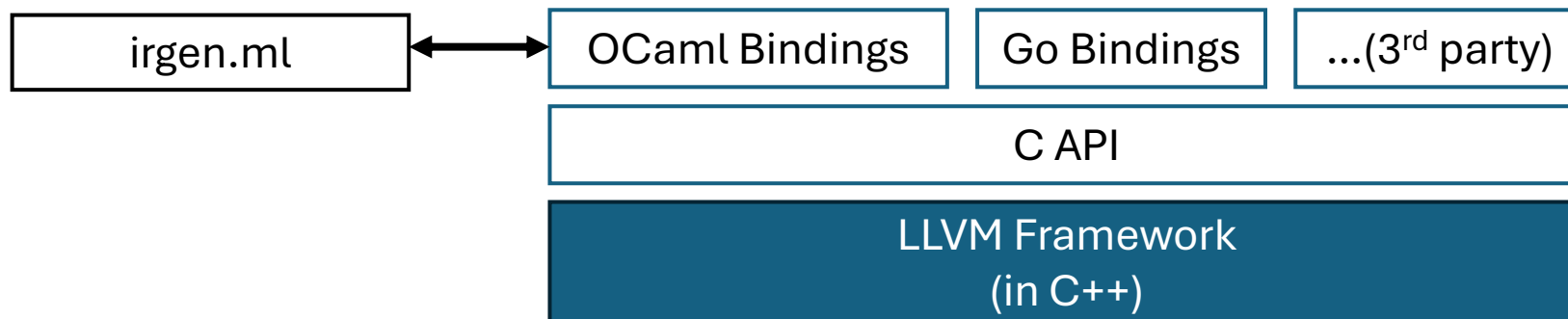
Fall 2026

Announcement

- Midterm on Monday Oct. 12, 10:30-12:00, at the classroom
 - + Additional time if required

Recap

- LLVM IR & Translation from RustY to LLVM IR
 - Modules, basic blocks, types, operations, instructions
 - Control flow, arrays, references
- Today: Instead of defining our own data types for LLVM IR, we will use the LLVM framework.
 - Through the official OCaml bindings:



Today's Topics

Constructing LLVM IR with the OCaml bindings

- ✓ LLVM objects, types, and instruction builders
- ✓ Memory, parameters, and function calls
- ✓ Basic blocks and conditional branches
- ✓ Short-circuit evaluation

(+ Tips for the midterm if time allows)

Construction of LLVM Objects through the API

Types for LLVM Objects

- Values of these OCaml types refer to C++ objects in LLVM.

```
module L = Llvm (* Alias for Llvm *)
```

<code>L.llcontext</code>	shared LLVM state
<code>L.llmodule</code>	a module
<code>L.lltype</code>	an LLVM type
<code>L.llvalue</code>	a value or instruction
<code>L.llbasicblock</code>	a basic block
<code>L.llbuilder</code>	an instruction builder

These types are abstract; their internal representation is hidden.

Context, Module, and Types

```
let ctx : L.llcontext = L.global_context ()  
let md : L.llmodule = L.create_module ctx "demo"
```

```
let i32t : L.lltype = L.i32_type ctx  
let i64t = L.i64_type ctx  
let i1t = L.i1_type ctx  
let voidt = L.void_type ctx  
let ptrt = L.pointer_type ctx
```

A context stores shared objects, such as LLVM types and constants.

- Repeated calls to `i32_type ctx` return the same LLVM type object.

Creating a Function

OCaml array notation

```
let fnty : L.lltype = L.function_type i32t [| i32t |]  
let fn : L.llvalue = L.define_function "add_one" fnty md  
let entry : L.llbasicblock = L.entry_block fn (*dflt: 'entry'*)  
  
let x : L.llvalue = L.param fn 0 (*unnamed*)  
let () = L.set_value_name "x" x
```



```
define i32 @add_one(i32 %0) {  
entry:  
    ↓  
    %x  
  
}
```

Building Instructions

```
let bld : L.llbuilder = L.builder ctx      (* Creates a builder *)
let () = L.position_at_end entry bld
let one : L.llvalue = L.const_int i32t 1
let res : L.llvalue = L.build_add x one "result" bld
                                     (* add's operand types must match *)
let () = ignore (L.build_ret res bld)
                                     (* ignore discards the return value *)
```

res refers to the add instruction **and**
represents its result (due to SSA).



```
define i32 @add_one(i32 %x) {
entry:
    %result = add i32 %x, 1
    ret i32 %result
}
```

Reading an API Signature

```
let sum = L.build_add x one "sum" bld    (* %sum = add i32 %x, 1 *)  
let v = L.build_load i32t addr "v" bld  (* %v = load i32, ptr %addr *)
```

```
L.build_add : L.llvalue -> L.llvalue -> string  
             -> L.llbuilder -> L.llvalue
```

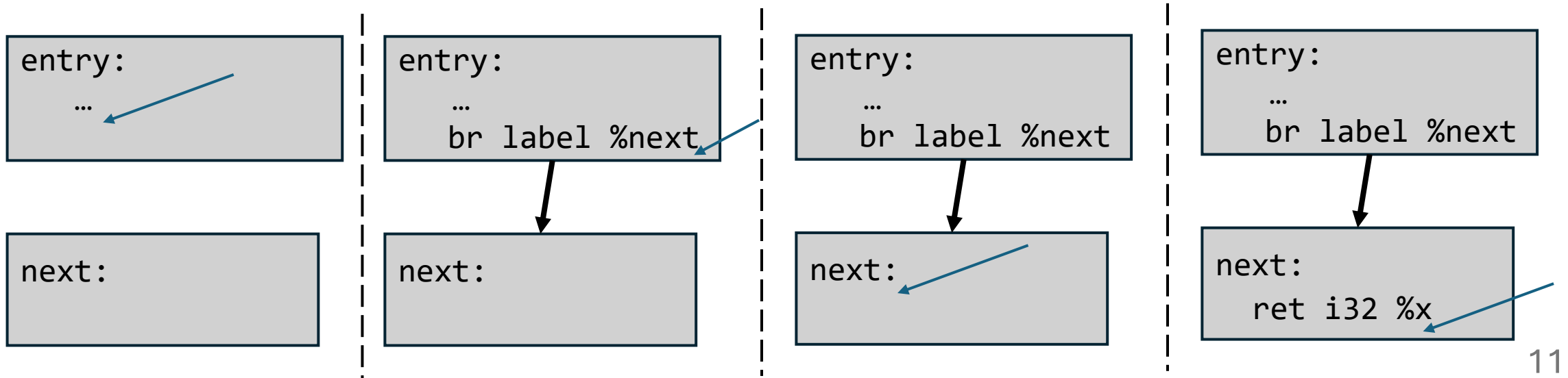
```
L.build_load : L.lltype -> L.llvalue -> string  
              -> L.llbuilder -> L.llvalue
```

- Operands are LLVM values.
- The builder supplies the position.
- After inserting an instruction, the builder is positioned immediately after it.
- A load also needs the type of the value to read.

The Builder Insertion Point

```
(* Start from an unterminated entry block. *)  
let next = L.append_block ctx "next" fn  
(* bld still points to entry. *)
```

```
let () = ignore (L.build_br next bld)  
let () = L.position_at_end next bld  
let () = ignore (L.build_ret x bld)
```



Memory Operations Through the API

```
let addr : L.llvalue = L.build_alloca i32t "x.addr" bld
let () = ignore (L.build_store one addr bld)
let v : L.llvalue = L.build_load i32t addr "v" bld
```



```
%x.addr = alloca i32
store i32 1, ptr %x.addr
%v = load i32, ptr %x.addr
```

Since the ‘store’ instruction is never used as an operand, we can safely ignore the returned `llvalue`.

Translation of Function Parameters

Storing Parameters in Memory

Parameter x arrives as an SSA value: %x.

Local variable y is read from memory using load.

Copy x into memory so that both can be read using load.

```
let xaddr = L.build_alloca i32t "x.addr" bld
let () = ignore (L.build_store x xaddr bld)
let v = L.build_load i32t xaddr "v" bld
```



```
%x.addr = alloca i32
store i32 %x, ptr %x.addr
%v = load i32, ptr %x.addr
```

Our compiler maps each variable name to its memory address.

Storing Parameters in Memory

- Store parameter values in memory to simplify IR generation.
 - To handle **parameters** and let-declared **variables** uniformly.
- Later optimizations can eliminate unnecessary memory operations.

```
fn sum(a: &i32, b: &i32) -> i32 {  
    return *a + *b;  
}  
  
fn f(x: i32) -> i32 {  
    let y: i32 = 1;  
    return sum(&x, &y);  
}
```

comp

```
define i32 @f(i32 %x) {  
entry:  
    %x.addr = alloca i32  
    %y.addr = alloca i32  
  
    store i32 %x, ptr %x.addr  
    store i32 1, ptr %y.addr  
  
    %result = call i32  
        @sum(ptr %x.addr, ptr %y.addr)  
    ret i32 %result  
}
```

Calling a Function

- Declare a function report and then call it.

```
let report_ty = L.function_type i1t [|||]  
let report_fn = L.declare_function "report" report_ty md  
let b = L.build_call report_ty report_fn [|||] "b" bld
```



```
declare i1 @report()
```

```
%b = call i1 @report()
```

Passing Arguments

```
let update_ty =  
  L.function_type voidt [| ptrt |]  
let update =  
  L.define_function "update" update_ty md  
let () = ignore (L.build_call update_ty  
  update [| naddr |] "" bld)
```



```
fn update(p: &mut i32) { ... }  
  
...  
update (&mut n);
```

comp

```
define void @update(ptr %p) {  
  %p.addr = alloca ptr  
  store ptr %p, ptr %p.addr  
  ...}  
  
call void @update(ptr %n.addr)
```

Attaching noalias to a Parameter

Our compiler marks &mut parameters with noalias.

- LLVM uses this promise for optimization.

```
declare void @update(ptr)
```

```
let attr : L.llattribute = L.create_enum_attr ctx "noalias" 0L  
let () = L.add_function_attr update attr (L.AttrIndex.Param 0)
```



```
declare void @update(ptr noalias)
```

The attribute belongs to the parameter, not the ptr type.

module AttrIndex
...
type t =
| Function
| Return
| Param of int
...

Translation of Arrays, Pointers, and Control Flow

Reading an Array Element

Read $a[i]$, where a contains four $i32$ elements.

```
let a4t = L.array_type i32t 4
let zero = L.const_int i64t 0
let elem = L.build_gep a4t addr [| zero; index |] "elem.addr" bld
let value = L.build_load i32t elem "value" bld
```



```
%elem.addr = getelementptr [4 x i32], ptr %a.addr,  
    i64 0, i64 %index  
%value = load i32, ptr %elem.addr
```

Why These APIs Need a Type

The ptr type does not specify a pointee type.

```
L.build_load i32t addr "value" bld
```

```
L.build_gep a4t addr [| zero; index |] "elem.addr" bld
```

```
L.build_call fnty fn [| x |] "result" bld
```

LLVM IR supports function pointers.

Branches and Terminators

Choose one of these instructions to end the current block.

```
(* Jump to next_bb. *)  
ignore (L.build_br next_bb bld)
```

```
(* Choose a destination using an i1 value. *)  
ignore (L.build_cond_br condition then_bb else_bb bld)
```

```
(* Return a value to the caller. *)  
ignore (L.build_ret result bld)
```

- The terminator must be the last instruction in its block.
- To fill another block, move the builder there first.

A Conditional Function

```
define i32 @choose(i1 %0) {  
entry:  
    br i1 %0,  
        label %then,  
        label %else  
  
then:  
    ret i32 10  
  
else:  
    ret i32 20  
}
```

```
let ty = L.function_type i32t [| i1t |] in  
let fn = L.define_function "choose" ty md in  
  
let tb = L.append_block ctx "then" fn in  
let fb = L.append_block ctx "else" fn in  
  
L.position_at_end (L.entry_block fn) bld;  
ignore (L.build_cond_br (L.param fn 0) tb fb bld);  
  
L.position_at_end tb bld;  
ignore (L.build_ret (L.const_int i32t 10) bld);  
  
L.position_at_end fb bld;  
ignore (L.build_ret (L.const_int i32t 20) bld)
```

Short-Circuiting Boolean Operations

(Which our compiler does not perform)

Short-Circuit Evaluation

A short-circuit operator may skip its right operand.

<code>a && b</code>	<code>a</code> is false: result is false, skip <code>b</code>
	<code>a</code> is true: evaluate <code>b</code>

<code>a b</code>	<code>a</code> is true: result is true, skip <code>b</code>
	<code>a</code> is false: evaluate <code>b</code>

Rust gives `&&` and `||` these short-circuit semantics.

When Evaluation Has Side Effects

```
fn report() -> bool {  
    println!("{}", 1);  
    return true;  
}
```

```
false && report()    // false, no output  
true  || report()    // true,  no output
```

With eager evaluation, `report()` runs in both cases.

```
%b = call i1 @report()  
%result = and i1 false, %b
```

Short-Circuiting with Branches

```
; a && report()
```

```
...    ; evaluation of a  
br i1 %a, label %and.rhs, label %false
```

```
and.rhs:
```

```
%b = call i1 @report()  
br i1 %b, label %true, label %false
```

```
true:
```

```
...
```

```
false:
```

```
...
```

For simplicity, RustY evaluates both operands of && and ||.

Checking IR Structure and Types

Verifying a Module

Verify after completing all function bodies.

```
let () = print_endline (L.string_of_llmodule md)
let () =
  match Llvm_analysis.verify_module md with
  | None -> L.print_module "demo.ll" md
  | Some message -> failwith message
```

It checks LLVM's structural and typing rules. For example:

- A basic block must end with a terminator.
- An i32 function cannot return an i1 value.